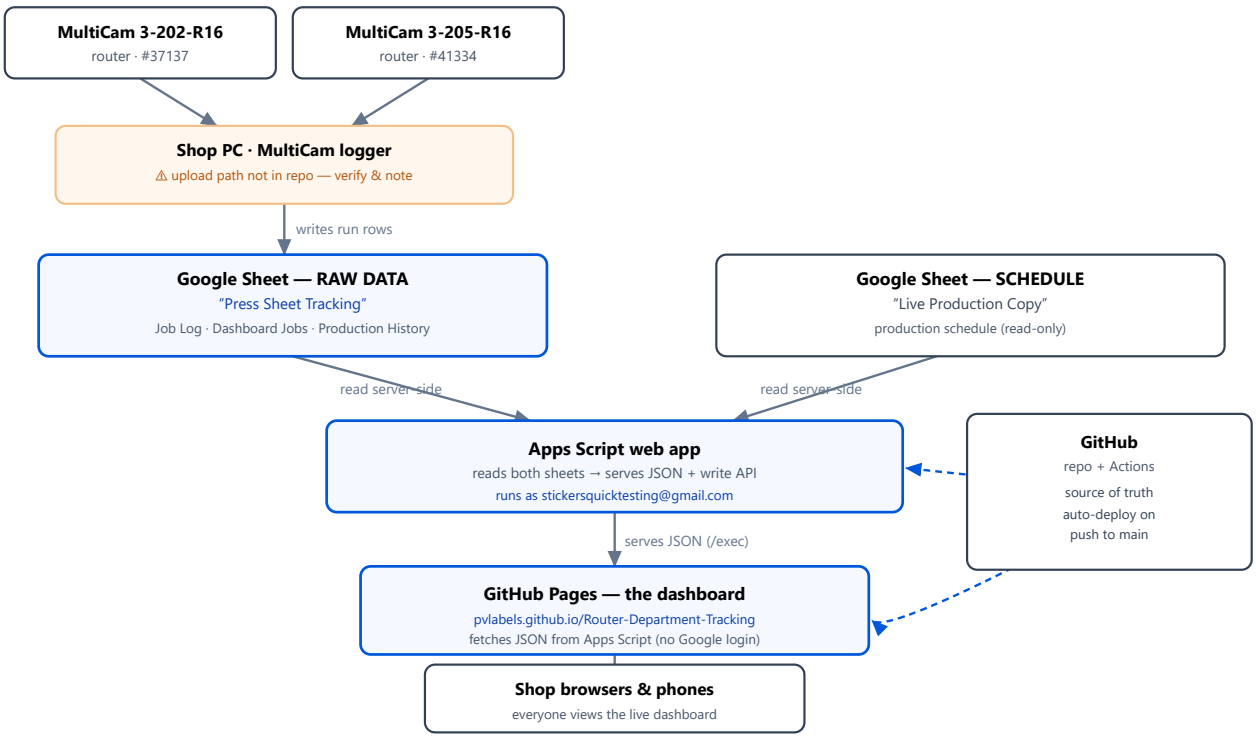
Router Department — System Map

How cut data flows from the machines to the dashboard — and what to check if something breaks.

Router Department Tracking
generated 2026-07-22



1 MultiCam routers (×2)

The two cutting machines. Names appear in the Job Log "Machine" column.

`MultiCam 3-202-R16 (37137)`
`MultiCam 3-205-R16 (41334)`

2 Shop PC · logger

Software on the router PC appends each finished cut (date, start/end, job, machine, status) as a row in the Job Log.

⚠ **Verify:** the exact logger software & upload method are not recorded here — confirm on the shop PC and write it in.

Note: its "Total Time" column runs +3 h high; the dashboard ignores it and uses End − Start.

3 Raw Data Sheet — "Press Sheet Tracking"

Where the machines write. Also holds the dashboard-owned tabs.

Owner: pv51@pvlabels.com · shared with stickersquicktesting@gmail.com
Tabs: Job Log (machine runs) · Dashboard Jobs · Production History
`1532iOs5fdQxJQyeQUatdDDq5jIEVKLYdYh-tRfXkXIc`
`docs.google.com/spreadsheets/d/1532iOs5…XkXIc/edit`

4 Schedule Sheet — "Live Production Copy"

Read by Apps Script for the queue, sizes, quantities, work orders, and the full-year history backfill.

Owner: pv51@pvlabels.com · shared (read) with stickersquicktesting@gmail.com
`1E0C4hanKBmYCrZw1V8DcknU60UxFtXAqMF48_XpDTik (Sheet1 / gid 0)`
`docs.google.com/spreadsheets/d/1E0C4h…DTik/edit`

5 Apps Script web app

Reads both sheets server-side; serves the dashboard JSON and the write API. Access: Anyone (anonymous).

Account (owns/runs): stickersquicktesting@gmail.com — the two sheets must stay shared with it.
`script:` `1J7W2WMz5ewvHzaH976p7eHy4nwX91c0ooHKz0Qc8t_XfrreYQT7aF6tO`
`deployment:` `AKfycbyO8LmIzyH1bw-axrpbrB98wsEVYZfCAJExR9vmBLoWcc9-WpvfsIhPW1FZ8Hl1dN1w3g`
`script.google.com/macros/s/AKfycby…/exec (add ?format=json)`

6 GitHub — source & auto-deploy

The real source of truth. Push to **main** → Actions pushes code to Apps Script (clasp), redeploys the web app, and publishes Pages.

`github.com/pvlabels/Router-Department-Tracking (public · acct pvlabels)`

Files: src/Code.js (backend) · src/index.html (dashboard) · .github/workflows/deploy.yml
Secret: CLASPRC_JSON (clasp token) · **Var:** DEPLOYMENT_ID = the #5 deployment · clasp pinned @2.4.2

7 GitHub Pages — the dashboard

The page everyone opens. Serves src/index.html and fetches JSON from #5 without Google cookies, so Workspace's block on script.google.com can't interfere.

`https://pvlabels.github.io/Router-Department-Tracking/`

Desktop shortcut "Router Dashboard.url" points here.

8 Local dev

Where the code is edited before pushing to GitHub.

`C:\Users\jperry\Desktop\AI Projects\Multicam Tracking`

Edit src/, commit, push to main — Actions does the rest. (This map: "Router Department System Map.html/.pdf".)

If something's down — start here

- **Dashboard blank or stale:** open …`/exec?format=json` (#5). If it errors, confirm sheets #3 and #4 are still shared with `stickersquicktesting@gmail.com`.
- **New cuts not appearing:** the shop logger (#2) isn't writing to the Job Log — check the router PC / logging software.
- **Queue, sizes, or WOs wrong/empty:** the schedule sheet (#4) moved, was renamed, or lost read access for the Apps Script account.
- **A code change didn't go live:** GitHub → **Actions** tab (#6) for a failed run. Pages OIDC hiccup → re-run with **"Run workflow" (workflow_dispatch)**, NOT "re-run failed jobs" (that duplicates the Pages artifact).
- **clasp / Apps Script login broke:** run `clasp login` with `@google/clasp@2.4.2`, then update the `CLASPRC_JSON` repo secret.

Details verified against the repo (README, .clasp.json, deploy.yml) and the live dashboard. The one item not documented in code — how the machines physically upload (#2) — is flagged for you to confirm.